Development of an Edge-Based AI Attendance Kiosk

An Industrial Internship Report

DITEC (Directorate of Information Technology, Electronics and Communication)

Biswajyoti Nath

July 2026

Abstract

This report details the architectural design and implementation of an enterprise-grade AI Attendance Kiosk, developed during an industrial internship at DITEC. The system modernizes legacy attendance tracking by utilizing a dual-authentication pipeline: QR-based identity fetching coupled with edge-based facial recognition. By processing AI inference strictly within browser Web Workers, the system achieves 60fps performance on commodity hardware without relying on costly external APIs. Furthermore, the architecture solves critical concurrency vulnerabilities (TOCTOU race conditions) using in-memory mutex locking, and secures administrative interfaces via Next.js Edge Middleware. The result is a robust, stateless containerized application that strictly enforces organizational time constraints.

Contents

Chapter 1

Introduction

Legacy attendance systems rely on vulnerable biometric scanners or easily spoofed ID cards. ”Buddy-punching” remains a significant operational challenge. The objective of this project was to design a touchless, highly secure kiosk that verifies physical presence in real-time.

1.1 Project Objectives

- Implement an edge AI facial recognition pipeline that runs independently of internet connectivity.
- Prevent duplicate attendance records during peak hours.
- Enforce strict organizational time policies (11:00 AM entry, 5:00 PM exit).
- Containerize the application for seamless deployment across DITEC hardware.

Chapter 2

System Architecture

The application follows a monolithic Next.js architecture, utilizing React for the interactive UI, Node.js for backend routing, and SQLite for persistent edge storage.

2.1 Edge AI Pipeline

Facial recognition algorithms are computationally expensive. Running them on the main browser thread causes unacceptable UI jitter. To resolve this, the `face-api.js` engine was isolated into a dedicated Web Worker. The main thread captures webcam frames at 60fps, downscales them to 320px, and passes the pixel buffer to the worker. The worker returns a 128-dimensional Float32Array descriptor which is compared against the database using Euclidean distance.

2.2 Concurrency Management

High-throughput environments are susceptible to Time-of-Check-to-Time-of-Use (TOCTOU) race conditions. If a user stands in front of the kiosk, the system might trigger multiple API requests simultaneously. This was mitigated by implementing an in-memory Mutex Lock (a JavaScript `Map`) at the API layer, instantly rejecting duplicate concurrent requests with an HTTP 409 Conflict.

Chapter 3

Security Implementation

Administrative endpoints require robust protection against unauthorized manipulation.

3.1 Edge Middleware Authentication

To secure the `/admin` dashboard and the underlying `/api/users` endpoints, HTTP Basic Authentication was implemented at the Edge level using Next.js Middleware. By intercepting unauthorized requests before they reach the React rendering engine, we ensure a zero-trust model. Credentials are dynamically injected via Docker environment variables, ensuring no secrets are hardcoded in the repository.

Chapter 4

Conclusion

The AI Attendance Kiosk successfully demonstrates the viability of executing complex AI models directly on edge hardware. By combining Web Workers, Mutex locks, and Edge Middleware, the system achieves enterprise-grade security and performance. This solution is production-ready and fully containerized, fulfilling all DITEC project requirements.